

- Andrews, G.E., Askey, R., and Roy, R. 1999, *Special Functions* (Cambridge, UK: Cambridge University Press).
- Thompson, W.J. 1997, *Atlas for Computing Mathematical Functions* (New York: Wiley-Interscience).
- Spanier, J., and Oldham, K.B. 1987, *An Atlas of Functions* (Washington: Hemisphere Pub. Corp.).
- Wolfram, S. 2003, *The Mathematica Book*, 5th ed. (Champaign, IL: Wolfram Media).
- Hart, J.F., et al. 1968, *Computer Approximations* (New York: Wiley).
- Hastings, C. 1955, *Approximations for Digital Computers* (Princeton: Princeton University Press).
- Luke, Y.L. 1975, *Mathematical Functions and Their Approximations* (New York: Academic Press).

6.1 Gamma Function, Beta Function, Factorials, Binomial Coefficients

The gamma function is defined by the integral

$$\Gamma(z) = \int_0^{\infty} t^{z-1} e^{-t} dt \quad (6.1.1)$$

When the argument z is an integer, the gamma function is just the familiar factorial function, but offset by 1:

$$n! = \Gamma(n + 1) \quad (6.1.2)$$

The gamma function satisfies the recurrence relation

$$\Gamma(z + 1) = z \Gamma(z) \quad (6.1.3)$$

If the function is known for arguments $z > 1$ or, more generally, in the half complex plane $\text{Re}(z) > 1$, it can be obtained for $z < 1$ or $\text{Re}(z) < 1$ by the reflection formula

$$\Gamma(1 - z) = \frac{\pi}{\Gamma(z) \sin(\pi z)} = \frac{\pi z}{\Gamma(1 + z) \sin(\pi z)} \quad (6.1.4)$$

Notice that $\Gamma(z)$ has a pole at $z = 0$ and at all negative integer values of z .

There are a variety of methods in use for calculating the function $\Gamma(z)$ numerically, but none is quite as neat as the approximation derived by Lanczos [1]. This scheme is entirely specific to the gamma function, seemingly plucked from thin air. We will not attempt to derive the approximation, but only state the resulting formula: For certain choices of rational γ and integer N , and for certain coefficients $c_1, c_2, \dots, c_N$, the gamma function is given by

$$\begin{aligned} \Gamma(z + 1) &= (z + \gamma + \frac{1}{2})^{z + \frac{1}{2}} e^{-(z + \gamma + \frac{1}{2})} \\ &\times \sqrt{2\pi} \left[c_0 + \frac{c_1}{z + 1} + \frac{c_2}{z + 2} + \dots + \frac{c_N}{z + N} + \epsilon \right] \quad (z > 0) \end{aligned} \quad (6.1.5)$$

You can see that this is a sort of take-off on Stirling's approximation, but with a series of corrections that take into account the first few poles in the left complex plane. The constant c_0 is very nearly equal to 1. The error term is parametrized by

ϵ . For $N = 14$, and a certain set of c 's and γ (calculated by P. Godfrey), the error is smaller than $|\epsilon| < 10^{-15}$. Even more impressive is the fact that, with these same constants, the formula (6.1.5) applies for the *complex* gamma function, *everywhere in the half complex plane* $\operatorname{Re} z > 0$, achieving almost the same accuracy as on the real line.

It is better to implement $\ln \Gamma(x)$ than $\Gamma(x)$, since the latter will overflow at quite modest values of x . Often the gamma function is used in calculations where the large values of $\Gamma(x)$ are divided by other large numbers, with the result being a perfectly ordinary value. Such operations would normally be coded as subtraction of logarithms. With (6.1.5) in hand, we can compute the logarithm of the gamma function with two calls to a logarithm and a few dozen arithmetic operations. This makes it not much more difficult than other built-in functions that we take for granted, such as $\sin x$ or e^x :

```
Doub gammln(const Doub xx) { gamma.h
Returns the value  $\ln[\Gamma(xx)]$  for  $xx > 0$ .
    Int j;
    Doub x,tmp,y,ser;
    static const Doub cof[14]={57.1562356658629235,-59.5979603554754912,
    14.1360979747417471,-0.491913816097620199,.339946499848118887e-4,
    .465236289270485756e-4,-.983744753048795646e-4,.158088703224912494e-3,
    -.210264441724104883e-3,.217439618115212643e-3,-.164318106536763890e-3,
    .844182239838527433e-4,-.261908384015814087e-4,.368991826595316234e-5};
    if (xx <= 0) throw("bad arg in gammln");
    y=x=xx;
    tmp = x+5.242187500000000000; Rational 671/128.
    tmp = (x+0.5)*log(tmp)-tmp;
    ser = 0.999999999999997092;
    for (j=0;j<14;j++) ser += cof[j]/++y;
    return tmp+log(2.5066282746310005*ser/x);
}
```

How shall we write a routine for the factorial function $n!$? Generally the factorial function will be called for small integer values, and in most applications the same integer value will be called for many times. It is obviously inefficient to call `exp(gammln(n+1.))` for each required factorial. Better is to initialize a static table on the first call, and do a fast lookup on subsequent calls. The fixed size 171 for the table is because $170!$ is representable as an IEEE double precision value, but $171!$ overflows. It is also sometimes useful to know that factorials up to $22!$ have exact double precision representations (52 bits of mantissa, not counting powers of two that are absorbed into the exponent), while $23!$ and above are represented only approximately.

```
Doub factrl(const Int n) { gamma.h
Returns the value  $n!$  as a floating-point number.
    static VecDoub a(171);
    static Bool init=true;
    if (init) {
        init = false;
        a[0] = 1.;
        for (Int i=1;i<171;i++) a[i] = i*a[i-1];
    }
    if (n < 0 || n > 170) throw("factrl out of range");
    return a[n];
}
```

More useful in practice is a function returning the log of a factorial, which doesn't have overflow issues. The size of the table of logarithms is whatever you can afford in space and initialization time. The value $\text{NTOP} = 2000$ should be increased if your integer arguments are often larger.

```
gamma.h  Doub factln(const Int n) {
Returns ln(n!).
    static const Int NTOP=2000;
    static VecDoub a(NTOP);
    static Bool init=true;
    if (init) {
        init = false;
        for (Int i=0;i<NTOP;i++) a[i] = gammaln(i+1.);
    }
    if (n < 0) throw("negative arg in factln");
    if (n < NTOP) return a[n];
    return gammaln(n+1.);          Out of range of table.
}
```

The binomial coefficient is defined by

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \quad 0 \leq k \leq n \quad (6.1.6)$$

A routine that takes advantage of the tables stored in `factrl` and `factln` is

```
gamma.h  Doub bico(const Int n, const Int k) {
Returns the binomial coefficient  $\binom{n}{k}$  as a floating-point number.
    if (n<0 || k<0 || k>n) throw("bad args in bico");
    if (n<171) return floor(0.5+factrl(n)/(factrl(k)*factrl(n-k)));
    return floor(0.5+exp(factln(n)-factln(k)-factln(n-k)));
    The floor function cleans up roundoff error for smaller values of n and k.
}
```

If your problem requires a series of related binomial coefficients, a good idea is to use recurrence relations, for example,

$$\begin{aligned} \binom{n+1}{k} &= \frac{n+1}{n-k+1} \binom{n}{k} = \binom{n}{k} + \binom{n}{k-1} \\ \binom{n}{k+1} &= \frac{n-k}{k+1} \binom{n}{k} \end{aligned} \quad (6.1.7)$$

Finally, turning away from the combinatorial functions with integer-valued arguments, we come to the beta function,

$$B(z, w) = B(w, z) = \int_0^1 t^{z-1} (1-t)^{w-1} dt \quad (6.1.8)$$

which is related to the gamma function by

$$B(z, w) = \frac{\Gamma(z)\Gamma(w)}{\Gamma(z+w)} \quad (6.1.9)$$

hence